Aarhus University

Department Of Computer Science

Compilation

Hand-in for Phase 3 (Individual Report)

Author
Søren M. Damsgaard

Student number
202309814

October 23, 2026



Task 1 - Implementing the Lexer

The lexer was implemneted in `lexer.mll` using ocamllex, regular expression are defined to see/recognize keywords, ops, literals ect... and also ignores/removes any whitespace and comments found.

Task 2 - Implementing the Parser

The parser was implemented in `parser.mly` using Menhir, that helps by mapping the token streams to an Ast structure.
This is done by letting Menhir recursively go through a list of statements that map tokens to Ast modules/structures, this works since Dolphin programs are just sequences of statements. it's also here that precedence and associativity are defined using `%nonassoc`, `%left` and `%right`, which should parse the syntax without shift/reduce conflicts as it is now.

Task 3 - Updating the Semantic Analysis

The updated `semant.ml` includes attached location parameter to each node, the location is used in the `errors.ml` module so that errors now include the line/column when encountering it and reporting it to the user.

Task 4 - Testing

The testing pipeline was yet again updated by implementing `compile_prog_from_filename`, which now combines the reading, lexing, parsing semantic analysis and the LLVM IR generation, and catches any errors underway, although right now it doesn't differentiate between errors in the parser or lexer, the frontend errors are generalized to the standard error type.

New negative tests were also added and older tests now also uses the new parser and lexer via `.dlp` files, and older tests were updated with location information, note that the locations are hardcoded for the assertions to better verifify and report errors.

How to run the code

You will have received a zip file containing the complete OCaml compiler source code, the Dune build configurations, and the C runtime environment.

This is run/compiled in VScode using the docker container provided.

To compile and run the project inside a VS Code development container, execute the following commands in the terminal while inside the folder:
The following commands is to run a test file manually

1. `dune build`
2. `dune exec dolphin -- compile --phase 3 tests/phase3/[name of test].dlp -o out.ll` (This runs and compiles a test, and outputs it into a file out.ll, prints error in terminal if any.)
3. `clang runtime.c out.ll -o ret`
4. `./ret` (outputs any prints into the terminal, like 'print_integer()')
5. `echo $?` (Outputs the result of the test)

The following commands is to run all the test suits using `OUnit2`

1. `dune build` (if not already doen)
2. `dune test` (runs the 'test_dolphin.ml' file)

The test file asserts each test case with the expected output, be it error messages or numerical output.